

Pico 10DOF IMU

From Waveshare Wiki
Jump to: navigation, search

Overview

The Pico-10DOF-IMU is an IMU sensor expansion module specialized for Raspberry Pi Pico. It incorporates sensors including a gyroscope, accelerometer, magnetometer, and baroreceptor, and uses an I2C bus for communication. Combined with the Raspberry Pi Pico, it can be used to collect environment sensing data like temperature and barometric pressure or to easily DIY a robot that detects motion gestures and orientations.

Feature

- Standard Raspberry Pi Pico header supports Raspberry Pi Pico series.
- Onboard MPU9250 (3-axis gyroscope, 3-axis accelerometer, and 3-axis magnetometer) for detecting motion gesture, orientation, and magnetic field.
- Onboard LPS22HB barometric pressure sensor, for sensing the atmospheric pressure of the environment.
- Provides online complete manual and resources (example programs such as Raspberry Pi Pico C/C++ and Micro Python).

Specifications

Sensor	Parameters
Acceslerometer	Resolution: 16 bits Measuring range (optional): ± 2 , ± 4 , ± 8 , $\pm 16g$ Operating current: 450uA
Gyroscope	Resolution: 16 bits Measuring range (optional): ± 250 , ± 500 , ± 1000 , $\pm 2000^{\circ}/sec$ Operating current: 3.2mA
Magnetometer	Resolution: 14 bits Measuring range: $\pm 4800\mu T$ (MPU9250) / $\pm 4900\mu T$ (ICM20948) Operating current: 280uA
Baroreceptor	Measuring range: 260 ~ 1260hPa Measuring accuracy (ordinary temperature): $\pm 0.025hPa$ Measuring speed: 1Hz - 75Hz
Electric	Parameters
Operating voltage	5V

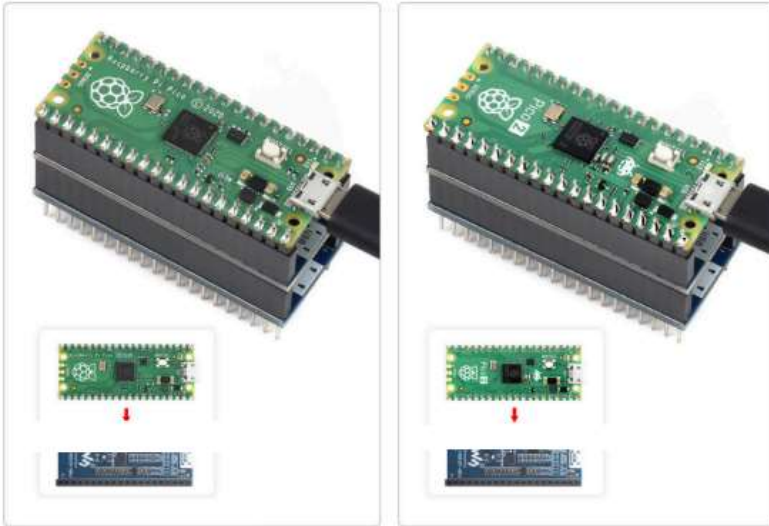
Hardware Description

- Pico-10DOF-IMU has three revisions:
 - 1. USB silkscreen update, add XYZ axis silkscreen and add 0R resistor for power management.
 - 2. The FSYNC, ICM.INT, and LPS.INT pins are respectively connected to two groups of GPIOs with 0-ohm resistors to avoid GPIO conflicts when sharing with other modules.
 - 3. Use MPU9250 instead of ICM20948, and change the silkscreen name to Pico-10DOF-IMU Rev2.1.



- 4. Reuse ICM20948, change silkscreen name to Pico-10DOF-IMU Rev3.1.

Hardware Connection



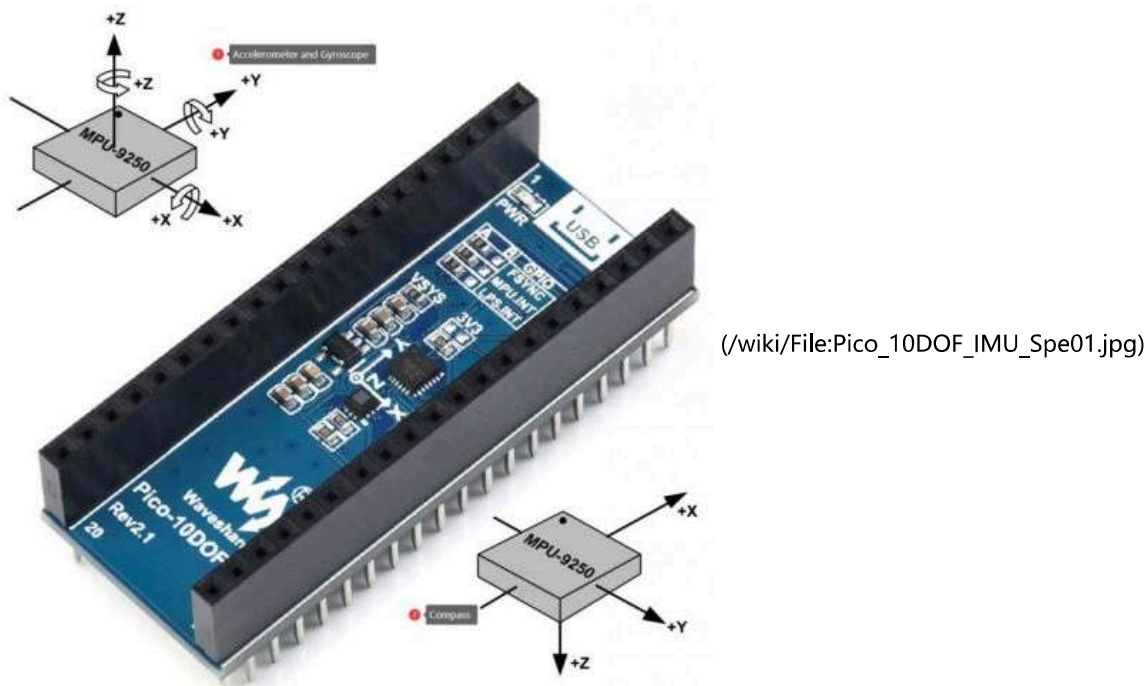
(/wiki/File:2024-09-10_115214.png)

1. Note that the USB Logo on the Pico-10DOF-IMU Rev2.1 corresponds to the USB connection direction of the Raspberry Pi Pico.
2. When downloading the C demo, be sure to press and hold the BOOT key before connecting the USB cable.

Axial Description

The axes of the accelerometer, gyroscope, and magnetometer on the MPU9250 are shown in the figure below. The magnetometer on the MPU9250 will be interfered with by hard magnetic, so when the data read by the magnetometer is fitted with an ellipsoid, the sphere is off-center and does not Circle. This will bring an initial magnetic field offset to the

magnetometer, making the magnetometer data eccentric. The magnetometer needs to be initialized when the power is turned on. Please refer to the initialization chapter below.



Pinout

GP0	1	40	VBUS
GP1	2	39	VSYS
GND	3	38	GND
GP2	4	37	3V3_EN
GP3	5	36	3V3(OUT)
GP4	6	35	ADC_VREF
GP5	7	34	GP28
GND	8	33	GND
GP6	9	32	GP27
GP7	10	31	GP26
GP8	11	30	RUN
GP9	12	29	GP22
GND	13	28	GND
GP10	14	27	GP21
GP11	15	26	GP20
GP12	16	25	GP19
GP13	17	24	GP18
GND	18	23	GND
GP14	19	22	GP17
GP15	20	21	GP16

VSYS	5V Power	
GND	Ground	
GP4	MPU.INT	MPU9250 INT pin
GP5	LPS.INT	LPS22HB INT pin
GP6	SDA	I2C SDA pin
GP7	SCL	I2C SCL pin
GP22	FSYNC	MPU9250 FSYNC pin

(/wiki/File:Pico-10DOF-IMU-details.jpg)

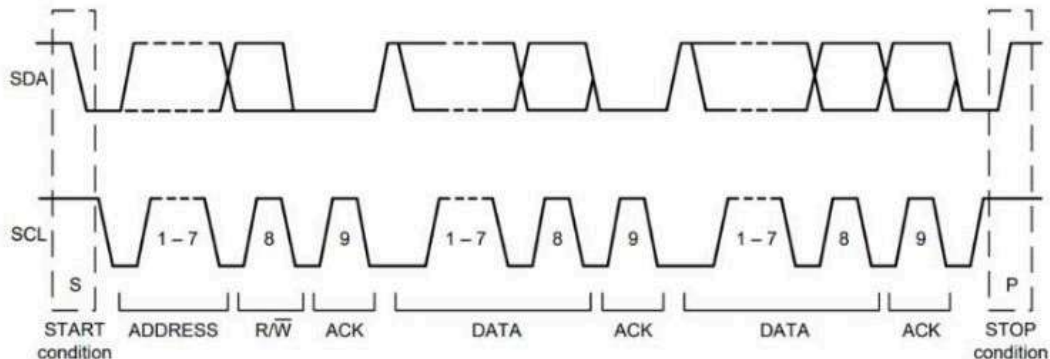
1. Pico-10DOF-IMU Rev2.1 uses GPIO as shown in the figure above, in which SDA (GPIO6), SCL (GPIO7) pins are fixedly connected, MPU9250 INT (GPIO4), FSYNC (GPIO22), LPS22HB INT (GPIO5) can be connected through 0R The resistance change connection pins are MPU9250 INT (GPIO22), FSYNC (GPIO16), LPS22HB INT (GPIO3), click to view the schematic diagram (https://files.waveshare.com/upload/e/ec/Pico-10DOF-IMU_Sch.pdf) for details.
2. The 40Pin of Pico-10DOF-IMU Rev2.1 is powered by the VSYS pin of Raspberry Pi Pico by default. If you want to turn off the 10DOS power supply, you can solder the 0R of R13 to R15 so that you can use the GPIO14 of Raspberry Pi Pico to turn on /off the 10DOF power supply. Please click to view the schematic diagram (https://files.waveshare.com/upload/e/ec/Pico-10DOF-IMU_Sch.pdf) for details.
3. If you want to use the 3.3V of Pico as the power supply, you can solder the 0R of R13 to R12. Please click to view the

schematic diagram (https://files.waveshare.com/upload/e/ec/Pico-10DOF-IMU_Sch.pdf) for details.

4. If you want to remove the LED of Pico-10DOF-IMU Rev2.1 and the 0R on the R11, you can click to view the schematic diagram (https://files.waveshare.com/upload/e/ec/Pico-10DOF-IMU_Sch.pdf) for details.

I2C Bus

- The Pico-10DOF-IMU onboard MPU9250 uses the I2C bus for communication. The read and write timing diagram is shown in the figure below. For more details, please refer to datasheet (<https://files.waveshare.com/upload/a/aa/MPU9250.PDF>).



Complete I²C Data Transfer

Single-Byte Write Sequence

Master	S	AD+W		RA		DATA		P
Slave			ACK		ACK		ACK	

Burst Write Sequence

Master	S	AD+W		RA		DATA		DATA		P
Slave			ACK		ACK		ACK		ACK	

Single-Byte Read Sequence

Master	S	AD+W		RA		S	AD+R			NACK	P
Slave			ACK		ACK			ACK	DATA		

Burst Read Sequence

Master	S	AD+W		RA		S	AD+R			ACK		NACK	P
Slave			ACK		ACK			ACK	DATA		DATA		

Signal	Description
S	Start Condition: SDA goes from high to low while SCL is high
AD	Slave I ² C address
W	Write bit (0)
R	Read bit (1)
ACK	Acknowledge: SDA line is low while the SCL line is high at the 9 th clock cycle
NACK	Not-Acknowledge: SDA line stays high at the 9 th clock cycle
RA	MPU-9250 internal register address
DATA	Transmit or received data
P	Stop condition: SDA going from low to high while SCL is high

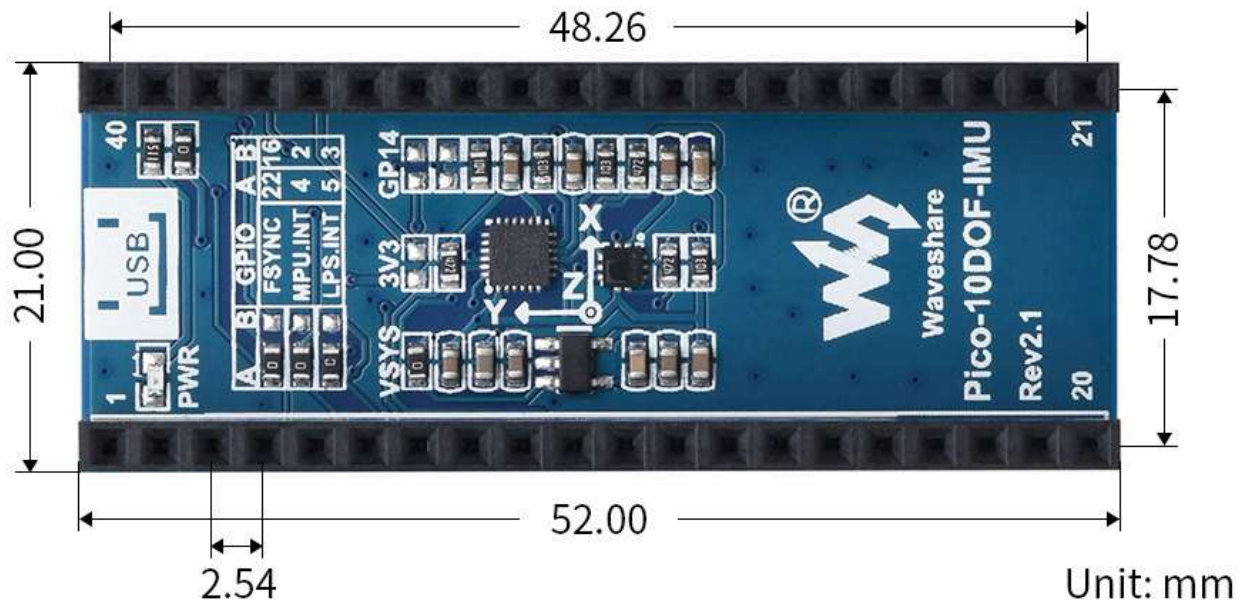
(/wiki/File:Pico_10DOF_IMU_Spec001.jpg)

- Raspberry Pi Pico, as the Master device, pulls down SDA successively, and the SCL pin initiates the START condition of the I2C bus, and then writes the device address (7bits) and write command (1bit) with a total of 8 bits of data. If the pin is

connected correctly, 10DOF is used as the slave device. Send an ACK response.

2. Raspberry Pi Pico continues to write the register address (RA) and register value (DATA) respectively and waits for the ACK response. After writing, the Raspberry Pi Pico pulls up SCL successively, and the SDA pin sends a STOP condition.
3. If the Raspberry Pi Pico reads the DATA of the register (RA), when writing RA and waiting for the ACK response, it re-initiates the START condition, and then writes the device address (7bits) and the read command (1bit) for a total of 8bits and waits for the ACK response. 10DOF returns to DATA. After Pico receives DATA, keep SDA high.
4. Please refer to the burst Read/Write Sequence in the figure above for the continuous write register value.

Outline Dimensions

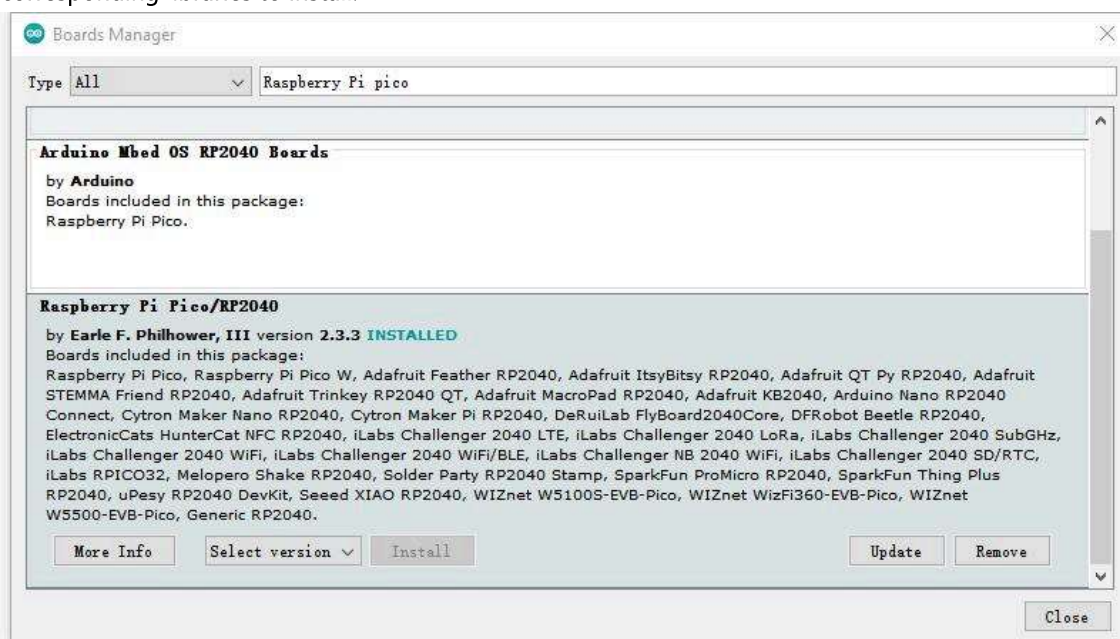


(/wiki/File:Pico-10DOF-IMU-details-size02.jpg)

Environment Building

- We test the code with Arduino IDE (<https://www.arduino.cc/>) and Thony (<https://thonny.org/>), click to download the related IDE, and open them after installation.

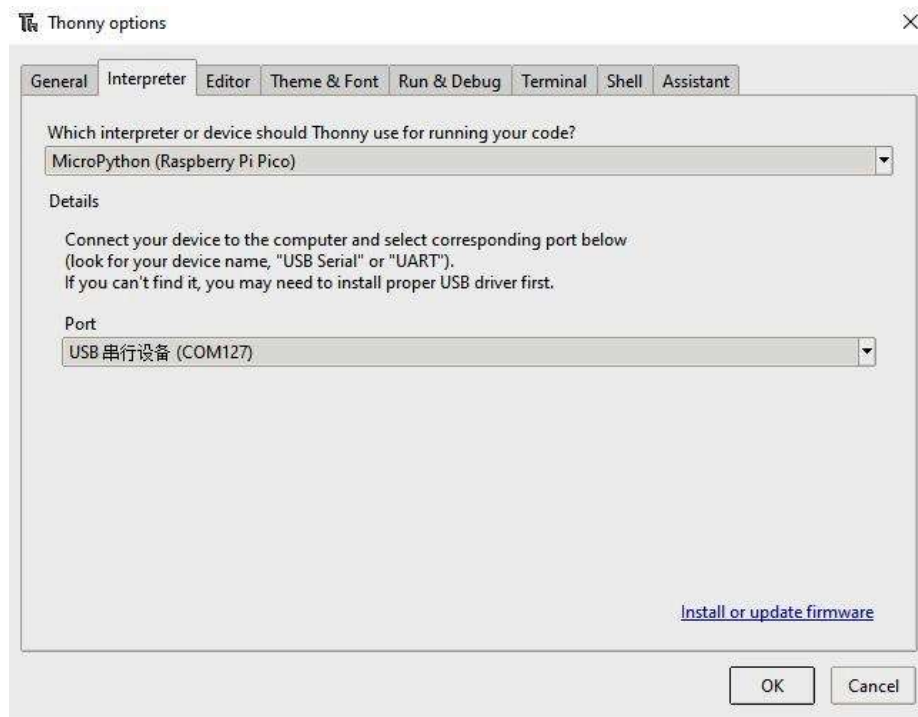
1. Install Pico SDK on Arduino IDE, click Tools->Board->Boards Manager, then search "Raspberry Pi Pico", and find the corresponding libraries to install.



(/wiki/File:Pico_10dof_imu_spec40.jpg)

2. Please refer to Micropython official document (<https://www.raspberrypi.com/documentation/microcontrollers/micropytho>)

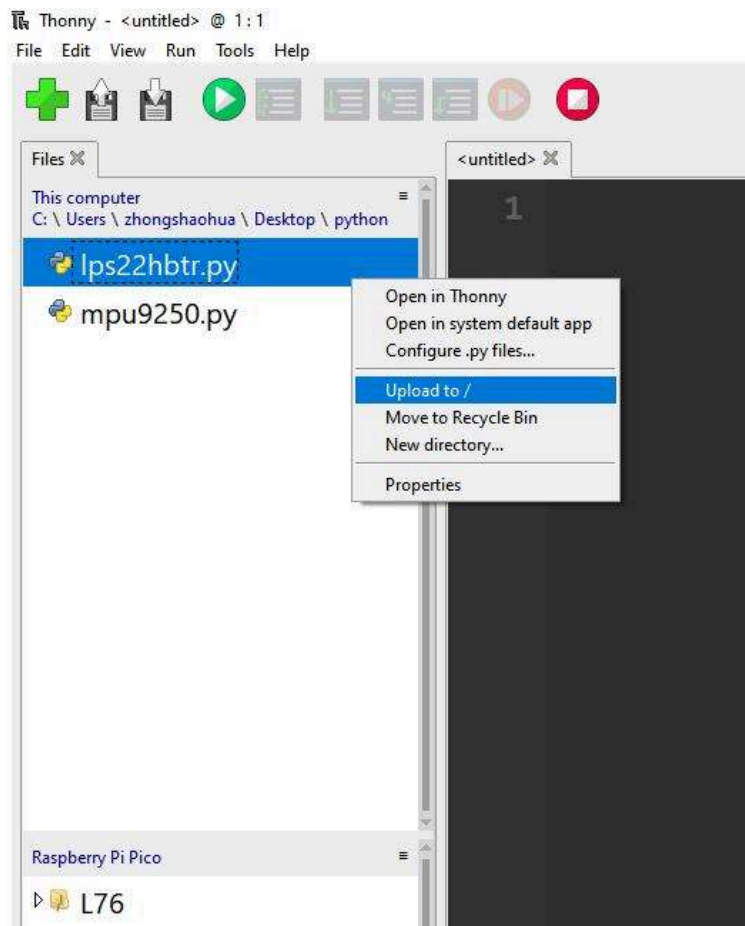
n.html) and set up python environment, select the Raspberry Pi Pico device in Thonny's Tools->Options->Interprete, as shown as below.



(/wiki/File:Pico-GPS-002.jpg)

Demo Download

1. Click to download sample demo (https://files.waveshare.com/upload/a/aa/Pico-10dof-imu_code.zip).
2. Unzip the sample demo, click .ino directly to open the Arduino sample demo, and upload the Micorpython sample demo to the Pico file system, as shown in the figure.



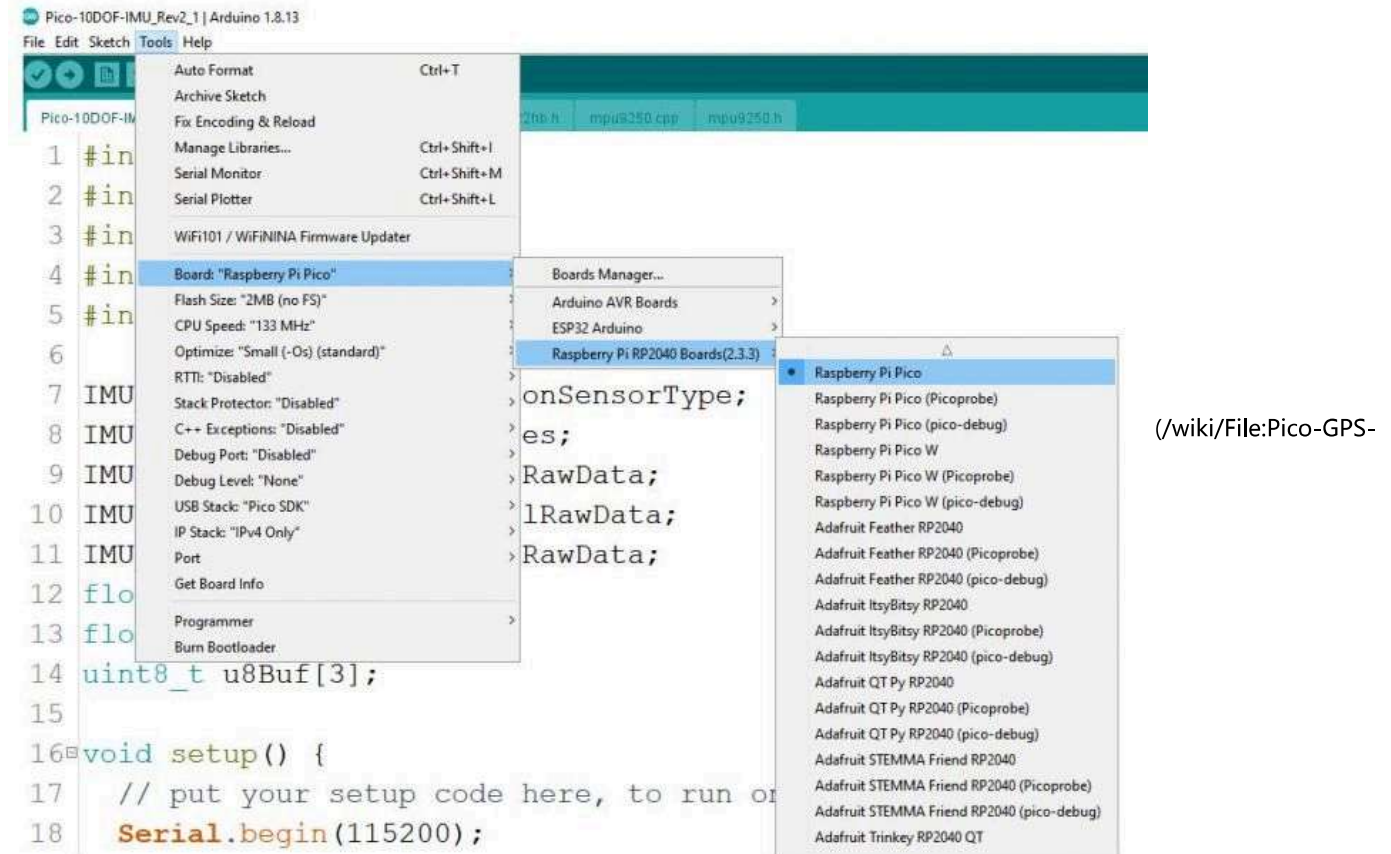
(/wiki/File:Thonny03.jpg)

Demo Usage

Arduino

- [Click here to install Arduino IDE](#)

1. Press and hold the BOOT button on the Pico and then connect the USB cable, open the .ino sample demo, click the menu bar, and select Tools -> Board -> Raspberry Pi Pico. As shown in the figure below.



003.jpg)

2. Click Upload under Edit to download the code to Pico. After downloading, open the device manager to view the virtual COM number of Pico, and then select the corresponding COM number in Tools -> Ports.

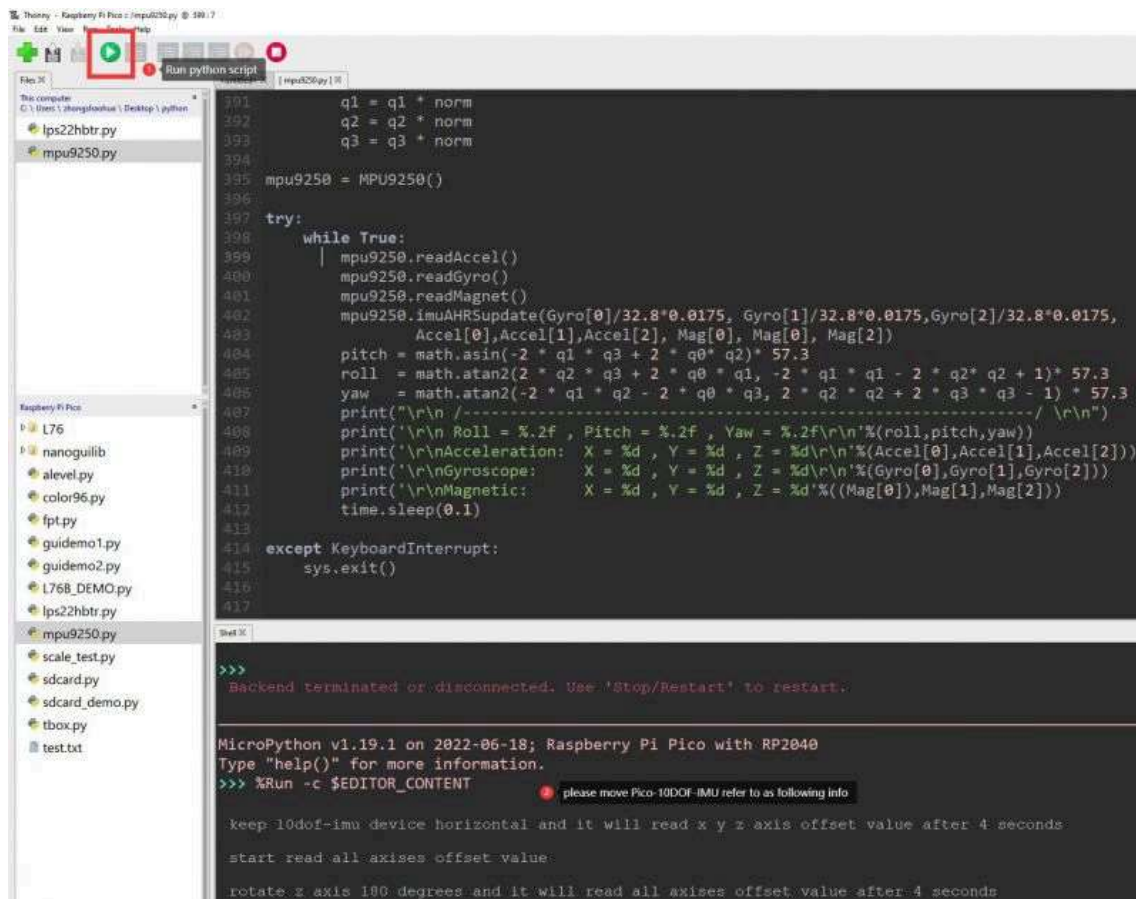
3. Click to open Monitor Serial at the top right of the Arduino IDE, and follow the serial port prompts to initialize the Pico-10DOF-IMU. For details, please refer to the Pico-10DOF-IMU initialization chapter.



004.jpg)

Micropython

- Please Install Thonny Python IDE (Windows V3.3.3) (<https://files.waveshare.com/upload/7/73/Thonny-3.3.3.zip>)
- Open the mpu9250.py, lps22hb.py scripts in the Pico file system, and click Run to run, where mpu9250.py will output relevant information to initialize the configuration of Pico-10DOF-IMU, as shown in the figure below, please refer to Pico-10DOF for the detailed process -IMU initialization chapter.



(/wiki/File:Pico-GPS-

005.jpg)

Initialization

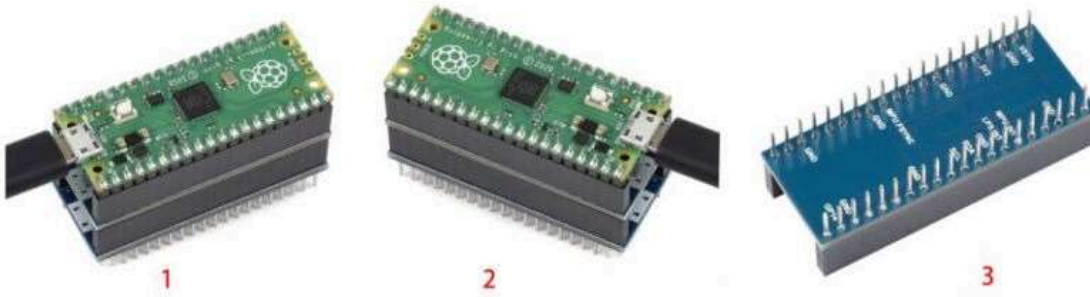
- The magnetometer on the MPU9250 will be interfered with by the hard magnetic field. When the ellipsoid fitting is performed on the data read by the magnetometer, the sphere is off-center and not round. This will bring an initial magnetic field offset to the magnetometer, making the magnetometer Data eccentricity, as shown in the figure below:



(/wiki/File:Pico-GPS-

006.png)

- After power-on, initialize according to the prompt information sent by the serial port. Calculate the eccentric offset value of the magnetometer, as shown in the following figure:



(/wiki/File:Pico-GPS-

```
MicroPython v1.19.1 on 2022-06-18; Raspberry Pi Pico with RP2040
Type "help()" for more information.
>>> %Run -c $EDITOR_CONTENT

1 keep 10dof-imu device horizontal and it will read x y z axis offset value after 4 seconds
  start read all axes offset value

2 rotate z axis 180 degrees and it will read all axes offset value after 4 seconds
  start read all axes offset value

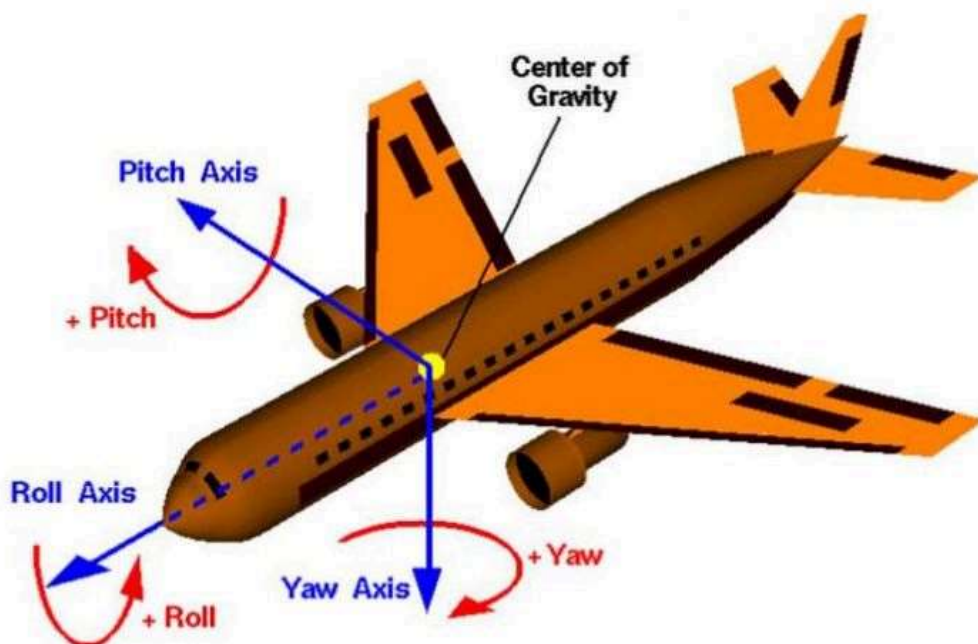
3 flip 10dof-imu device and keep it horizontal and it will read all axes offset value after 4 seconds
  start read all axes offset value
```

007.jpg)

Demo Analysis

This section briefly analyzes the mpu9250.py sample program.

- `mpu9250 = MPU9250()` instantiates the MPU9250 class. The instantiation process will include gyroscope initialization and geomagnetic calibration.
- `mpu9250.readAccel()`, `mpu9250.readGyro()`, `mpu9250.readMagnet()` respectively read raw data such as accelerometer, gyroscope, geomagnetometer, etc.
- `mpu9250.imuAHRSupdate()` is the mahony algorithm used to convert the values of the accelerometer, gyroscope and geomagnetometer into Euler angles (pitch, roll, yaw). Please click the link (<https://ahrs.readthedocs.io/en/latest/filters/mahony.html>) to view the detailed process of the mahony algorithm
- Euler angles are shown in the figure below:



(/wiki/File:Pico-GPS-

008.jpg)

```

mpu9250 = MPU9250()

try:
    while True:
        mpu9250.readAccel()
        mpu9250.readGyro()
        mpu9250.readMagnet()
        mpu9250.imuAHRUpdate(Gyro[0]/32.8*0.0175, Gyro[1]/32.8*0.0175,Gyro[2]/32.8*0.0175,
                             Accel[0],Accel[1],Accel[2], Mag[0], Mag[0], Mag[2])
        pitch = math.asin(-2 * q1 * q3 + 2 * q0* q2)* 57.3
        roll  = math.atan2(2 * q2 * q3 + 2 * q0 * q1, -2 * q1 * q1 - 2 * q2* q2 + 1)* 57.3
        yaw   = math.atan2(-2 * q1 * q2 - 2 * q0 * q3, 2 * q2 * q2 + 2 * q3 * q3 - 1) * 57.3
        print("\r\n /-----/ \r\n")
        print('\r\n Roll = %.2f , Pitch = %.2f , Yaw = %.2f\r\n'%(roll,pitch,yaw))
        print('\r\nAcceleration:  X = %d , Y = %d , Z = %d\r\n'%(Accel[0],Accel[1],Accel[2]))
        print('\r\nGyroscope:     X = %d , Y = %d , Z = %d\r\n'%(Gyro[0],Gyro[1],Gyro[2]))
        print('\r\nMagnetic:      X = %d , Y = %d , Z = %d'%(Mag[0]),Mag[1],Mag[2]))
        time.sleep(0.1)

except KeyboardInterrupt:
    sys.exit()

```

Resource

Documents

- Rev2.1 Schematic Diagram (https://files.waveshare.com/upload/f/f1/Pico-10DOF-IMU_Sch2.pdf)
- Rev3.1 Schematic Diagram (https://files.waveshare.com/upload/f/f1/Pico-10DOF-IMU_Rev3.1_Sch.pdf)
- ICM20948 Datasheet (<https://files.waveshare.com/upload/5/57/ICM-20948-v1.3.pdf>)
- LPS22HB Specification (<https://files.waveshare.com/upload/2/20/Lps22hb.pdf>)
- LSF0204d Specification (<https://files.waveshare.com/upload/c/cc/Lsf0204d.pdf>)
- MPU9250 (<https://files.waveshare.com/upload/a/aa/MPU9250.PDF>)

Demo Codes

- Demo code (https://files.waveshare.com/upload/a/aa/Pico-10dof-imu_code.zip)
 - Rev3.1 shared demo with Rev1.0.

Development Software

- Zimo221.7z (<https://files.waveshare.com/upload/c/c6/Zimo221.7z>)
- Image2Lcd.7z (<https://files.waveshare.com/upload/3/36/Image2Lcd.7z>)
- Font Library Tutorial (https://www.waveshare.com/wiki/Ink_Screen_Font_Library_Tutorial)
- Image Extraction Tutorial (https://www.waveshare.com/wiki/Image_Extraction)
- Thonny Python IDE (Windows V3.3.3) (<https://files.waveshare.com/upload/7/73/Thonny-3.3.3.zip>)

Pico Getting Started

Firmware Download

- MicroPython Firmware Download

[\[Expand\]](#)

Introduction

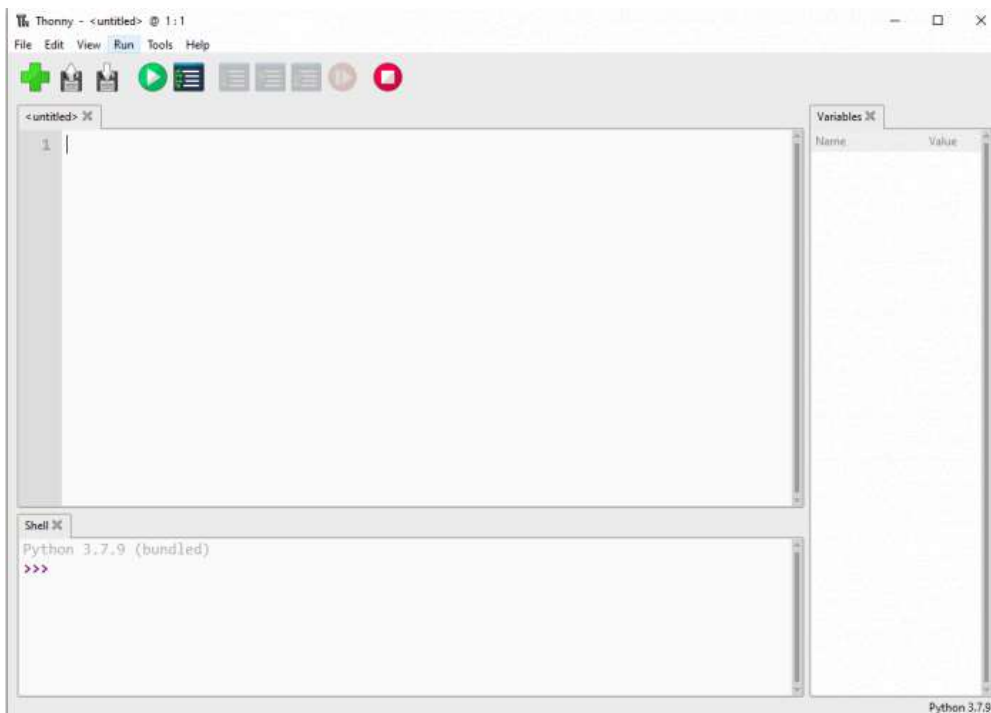
Raspberry Pi Pico Basics (https://www.waveshare.com/wiki/Raspberry_Pi_Pico)

MicroPython Series

Install Thonny IDE

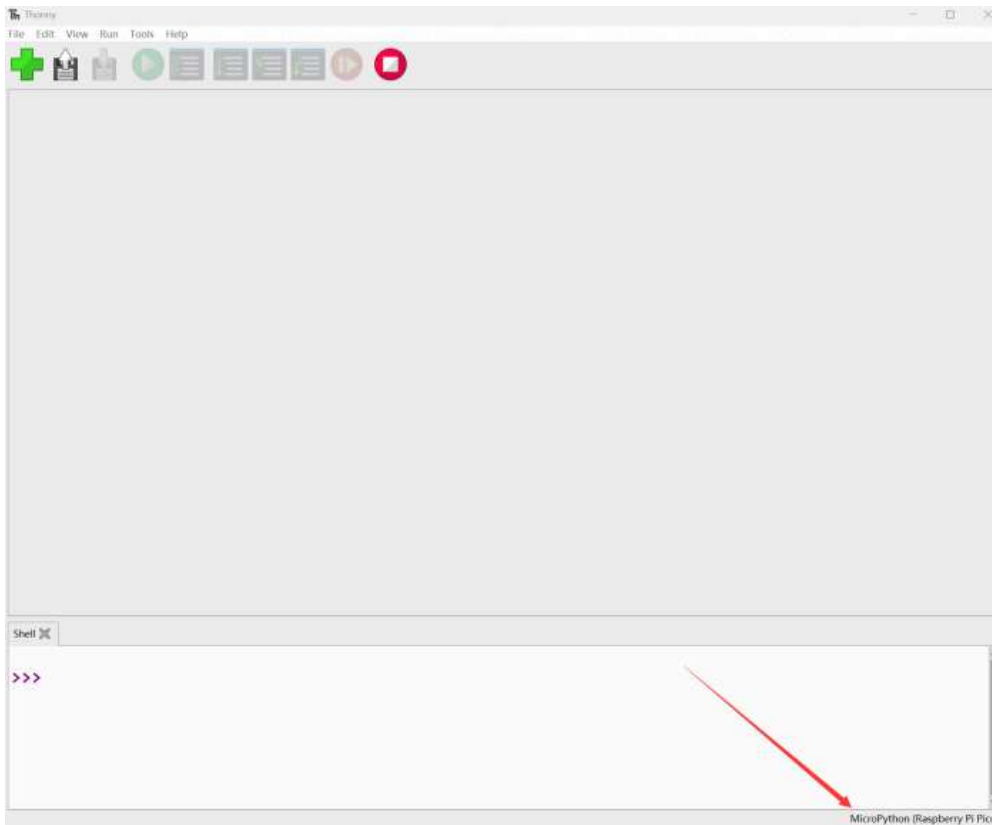
In order to facilitate the development of Pico/Pico2 boards using MicroPython on a computer, it is recommended to download the Thonny IDE

- Download Thonny IDE and follow the steps to install, the installation packages are all Windows versions, please refer to Thonny's official website for other versions
 - Thonny IDE official download link (<https://github.com/thonny/thonny/releases/download/v3.3.3/thonny-3.3.3.exe>)
 - Thonny IDE download link (<https://files.waveshare.com/wiki/common/Thonny-3.3.3.zip>)
 - Thonny official website (<https://thonny.org/>)
- After installation, the language and motherboard environment need to be configured for the first use. Since we are using Pico/Pico2, pay attention to selecting the Raspberry Pi option for the motherboard environment



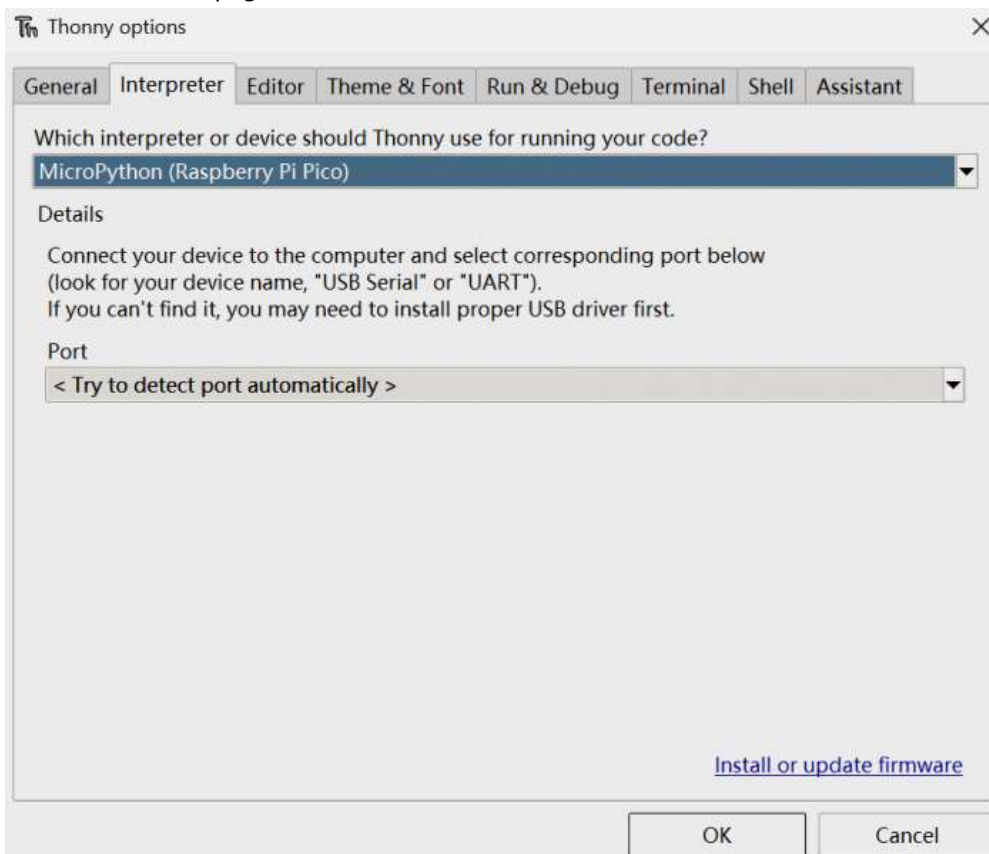
(/wiki/File:Pico-R3-Tonny1.png)

- Configure MicroPython environment and choose Pico/Pico2 port
 - Connect Pico/Pico2 to your computer first, and in the lower right corner of Thonny left-click on the configuration environment option --> select Configure interpreter
 - In the pop-up window, select MicroPython (Raspberry Pi Pico), and choose the corresponding port



(/wiki/File:700px-Raspberry-Pi-

Pico-Basic-Kit-M-2.png)



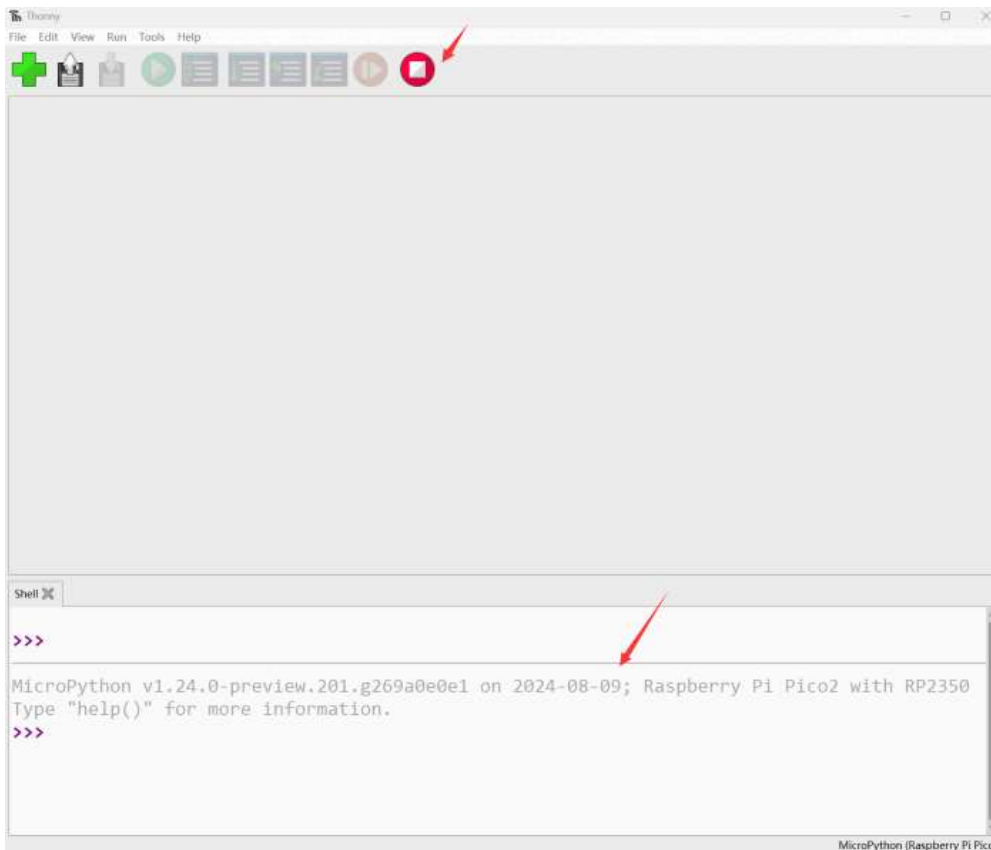
(/wiki/File:700px-Raspberry-Pi-

Pico-Basic-Kit-M-3.png)

Flash Firmware

- Click OK to return to the Thonny main interface, download the corresponding firmware library and burn it to the device, and then click the Stop button to display the current environment in the Shell window
- **Note: Flashing the Pico2 firmware provided by Micropython may cause the device to be unrecognized, please use the firmware below or in the package**
 - Pico firmware library (<https://files.waveshare.com/wiki/common/Rp2-pico-20210418-v1.15.7z>)
 - Pico2 firmware library (https://files.waveshare.com/wiki/common/RPI_PICO2-20240809-v1.24.0.zip)
- How to download the firmware library for Pico/Pico2 in windows: After holding down the BOOT button and connecting to the computer, release the BOOT button, a removable disk will appear on the computer, copy the firmware library into it

- How to download the firmware library for RP2040/RP2350 in windows: After connecting to the computer, press the BOOT key and the RESET key at the same time, release the RESET key first and then release the BOOT key, a removable disk will appear on the computer, copy the firmware library into it (you can also use the Pico/Pico2 method)



(/wiki/File:Raspberry-Pi-Pico2-

Python.png)

MicroPython Series

【MicroPython】 machine.Pin class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91Machine.Pin_Functions)

【MicroPython】 machine.PWM class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.PWM_Function)

【MicroPython】 machine.ADC class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.ADC_Function)

【MicroPython】 machine.UART class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.UART_Function)

【MicroPython】 machine.I2C class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.I2C_Function)

【MicroPython】 machine.SPI class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91machine.SPI_Function)

【MicroPython】 rp2.StateMachine class function details (https://www.waveshare.com/wiki/%E3%80%90MicroPython%E3%80%91rp2.StateMachine_Function)

C/C++ Series

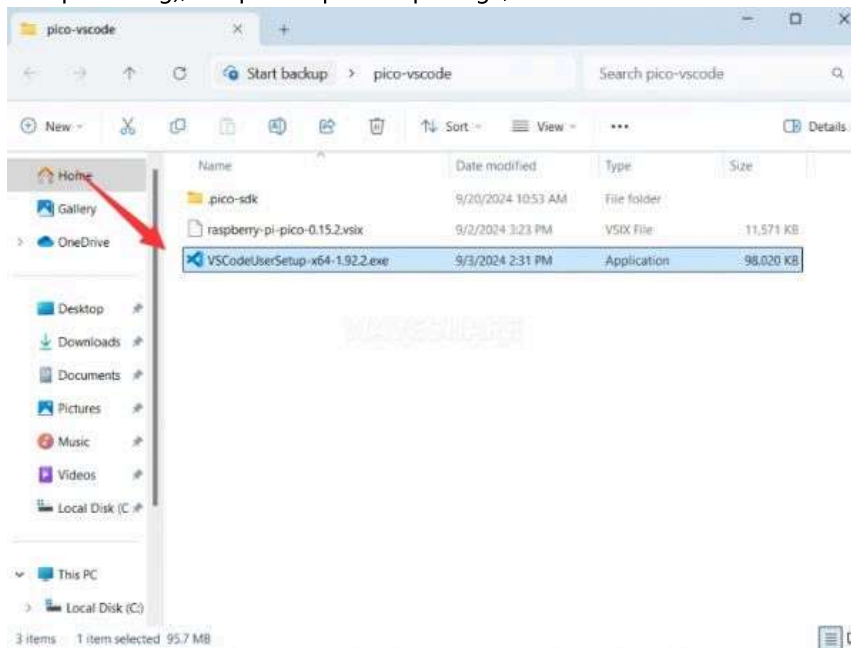
For C/C++, it is recommended to use Pico VS Code for development. This is a Microsoft Visual Studio Code extension designed to make it easier for you to create, develop, and debug projects for the Raspberry Pi Pico series development boards. No matter if you are a beginner or an experienced professional, this tool can assist you in developing Pico with confidence and ease. Here's how to install and use the extension.

- Official website tutorial: <https://www.raspberrypi.com/news/pico-vscode-extension/> (<https://www.raspberrypi.com/news/pico-vscode-extension/>)
- This tutorial is suitable for Raspberry Pi Pico, Pico2 and the RP2040 and RP2350 series development boards developed by Waveshare

- The development environment defaults to Windows11. For other environments, please refer to the official tutorial for installation

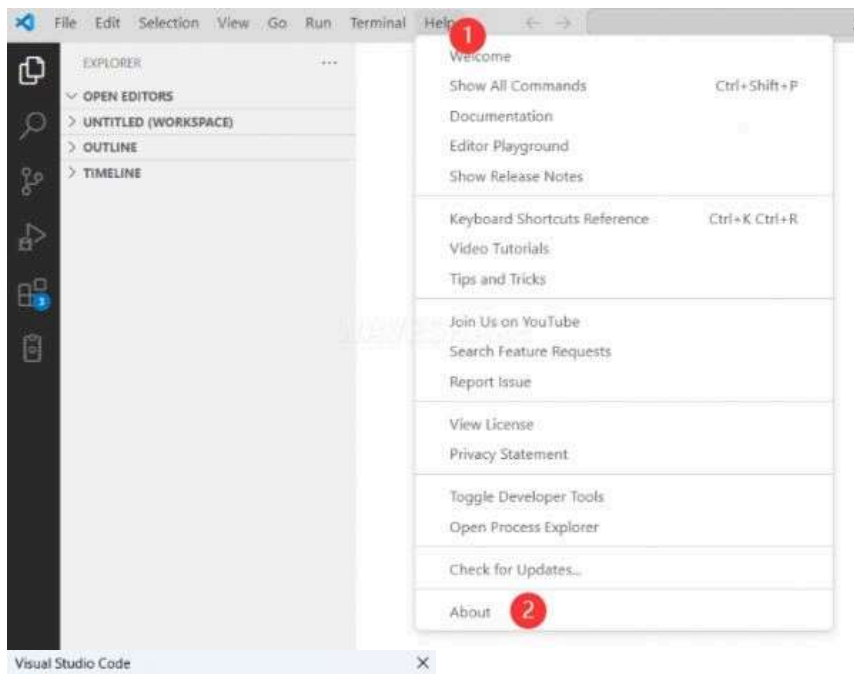
Install VSCode

1. First, click to download pico-vscode package (<https://drive.google.com/file/d/18-KDNrQII0KuTMdS6W5ibIUgaGm3FbVJ/view?usp=sharing>), unzip and open the package, double-click to install VSCode

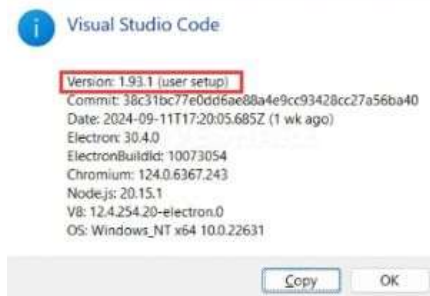


(/wiki/File:Pico-vscode-1.JPG)

Note: If vscode is installed, check if the version is v1.87.0 or later



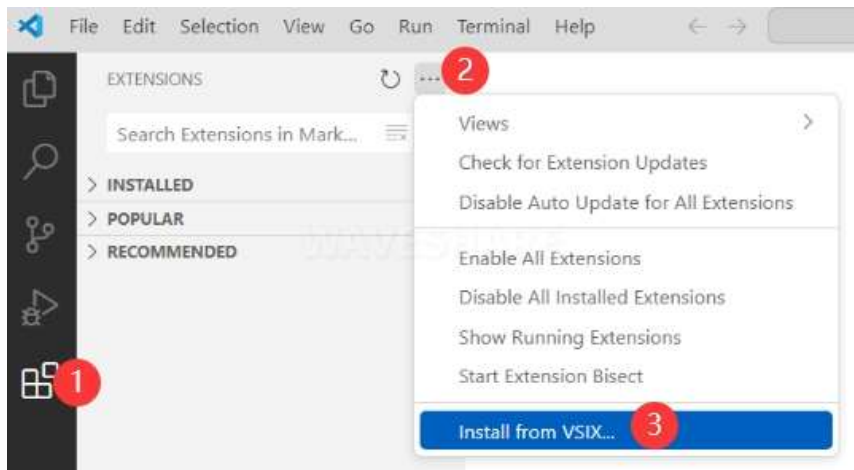
(/wiki/File:Pico-vscode-2.JPG)



(/wiki/File:Pico-vscode-3.JPG)

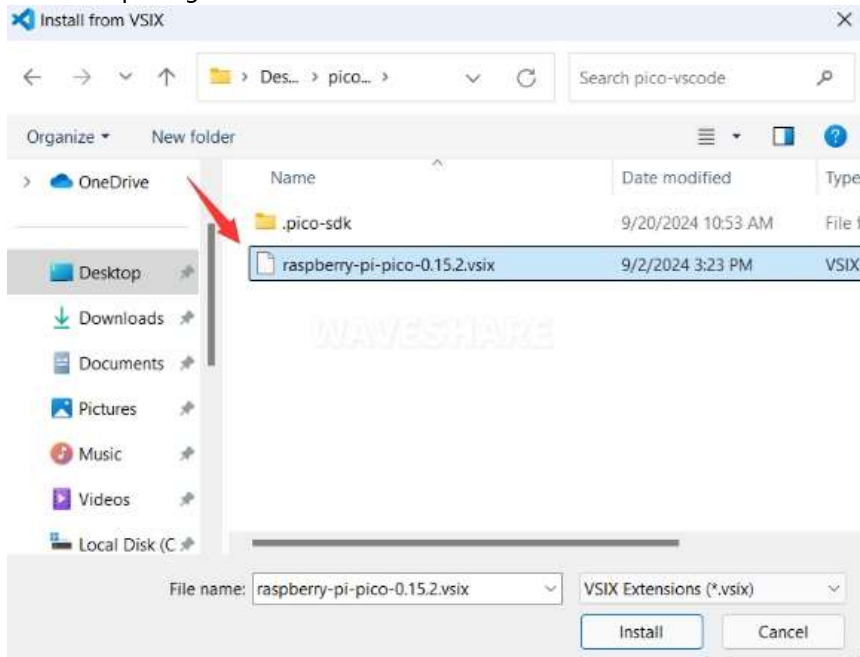
Install Extension

1. Click Extensions and select Install from VSIX



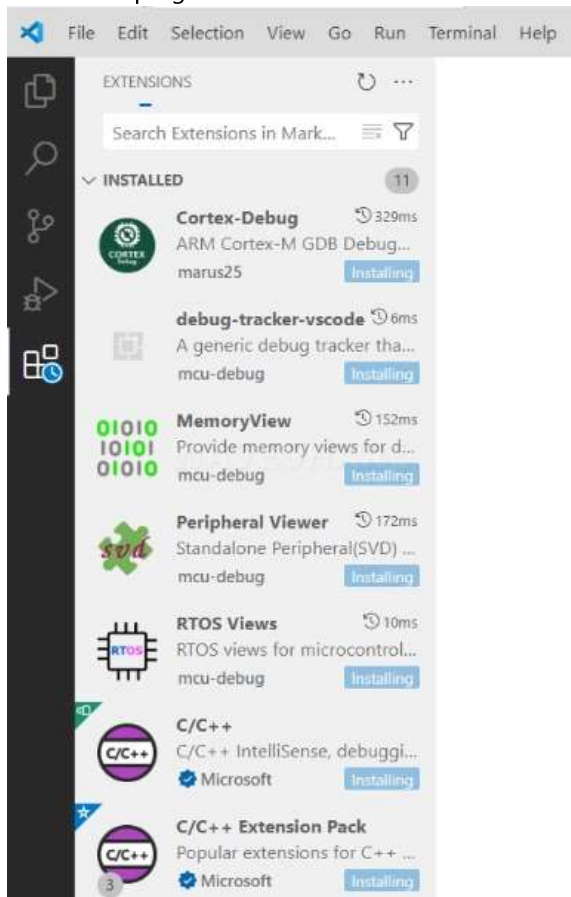
(/wiki/File:Pico-vscode-4.JPG)

2. Select the package with the vsix suffix and click Install



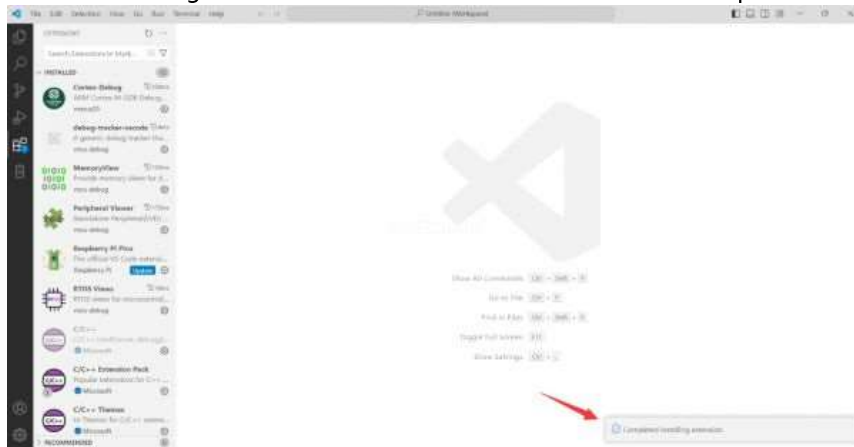
(/wiki/File:Pico-vscode-5.JPG)

3. Then vscode will automatically install raspberrypi-pico and its dependency extensions, you can click Refresh to check the installation progress



(/wiki/File:Pico-vscode-6.JPG)

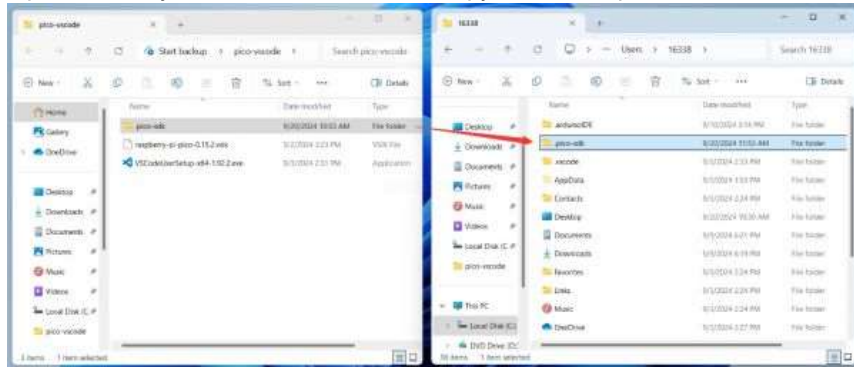
4. The text in the right lower corner shows that the installation is complete. Close VSCode



(/wiki/File:Pico-vscode-7.JPG)

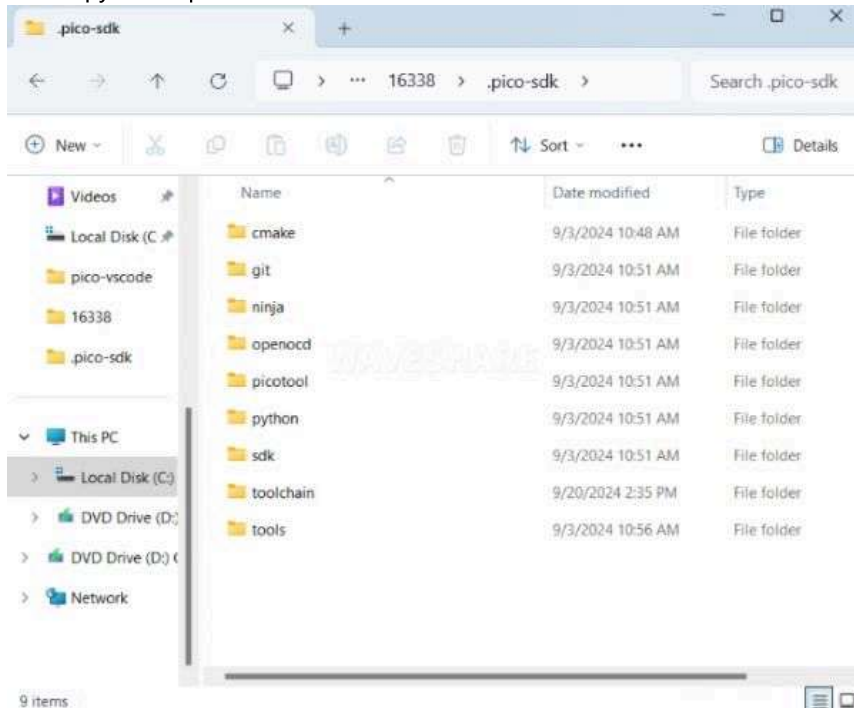
Configure Extension

1. Open directory C:\Users\username and copy the entire .pico-sdk to that directory



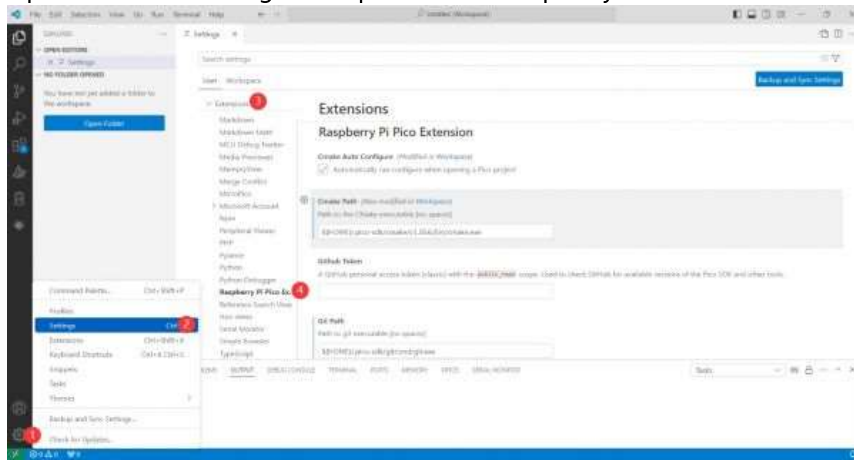
(/wiki/File:Pico-vscode-8.JPG)

2. The Copy is completed



(/wiki/File:Pico-vscode-9.JPG)

3. Open vscode and configure the paths for the Raspberry Pi Pico extensions



(/wiki/File:Pico-vscode-10.JPG)

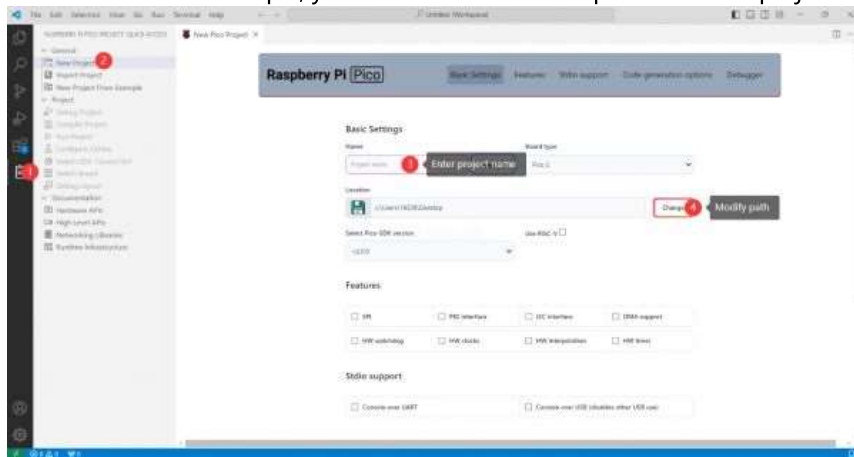
The configuration is as follows:

```
Cmake Path:  
${HOME}/.pico-sdk/cmake/v3.28.6/bin/cmake.exe  
  
Git Path:  
${HOME}/.pico-sdk/git/cmd/git.exe  
  
Ninja Path:  
${HOME}/.pico-sdk/ninja/v1.12.1/ninja.exe  
  
Python3 Path:  
${HOME}/.pico-sdk/python/3.12.1/python.exe
```

New Project

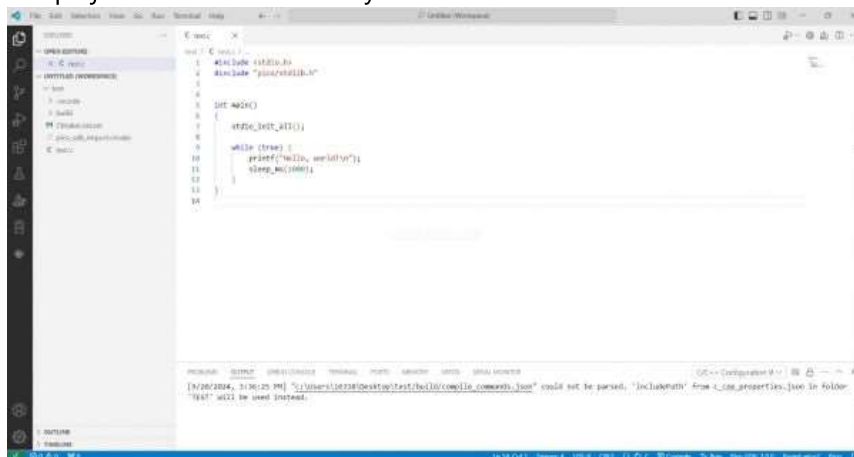
1. The configuration is complete, create a new project, enter the project name, select the path, and click Create to create the project

To test the official example, you can click on the Example next to the project name to select



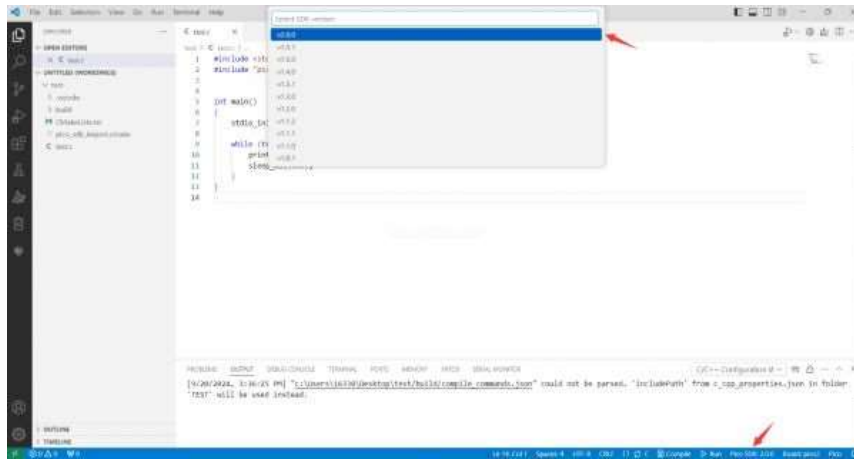
(/wiki/File:Pico-vscode-11.JPG)

2. The project is created successfully



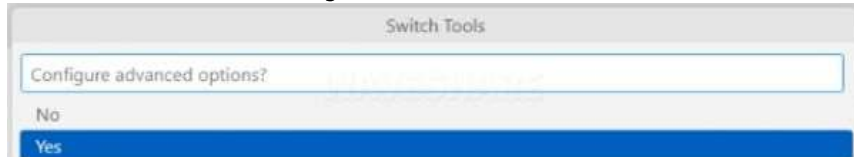
(/wiki/File:Pico-vscode-12.JPG)

3. Select the SDK version



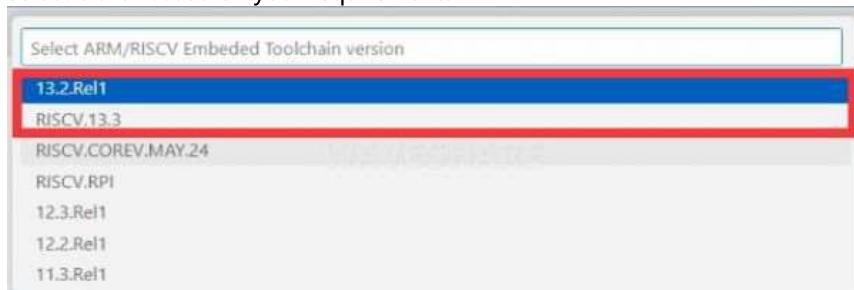
(/wiki/File:Pico-vscode-13.JPG)

4. Select Yes for advanced configuration



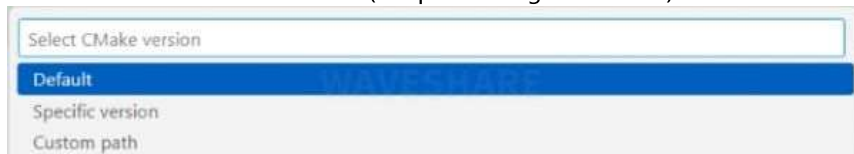
(/wiki/File:Pico-vscode-14.JPG)

5. Choose the cross-compilation chain, 13.2.Rel1 is applicable for ARM cores, RISC.V13.3 is applicable for RISC.V cores. You can select either based on your requirements



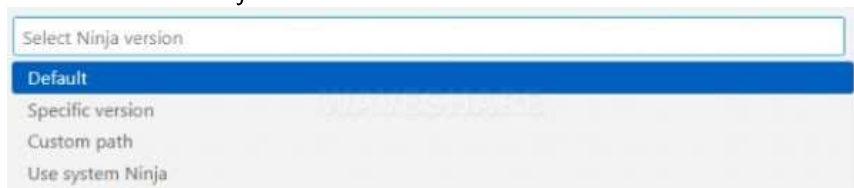
(/wiki/File:Pico-vscode-15.JPG)

6. Select default for CMake version (the path configured earlier)



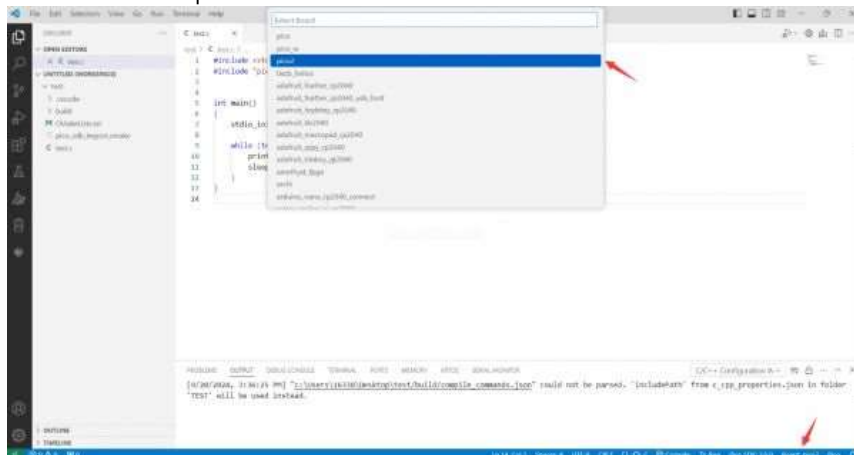
(/wiki/File:Pico-vscode-16.JPG)

7. Select default for Ninja version



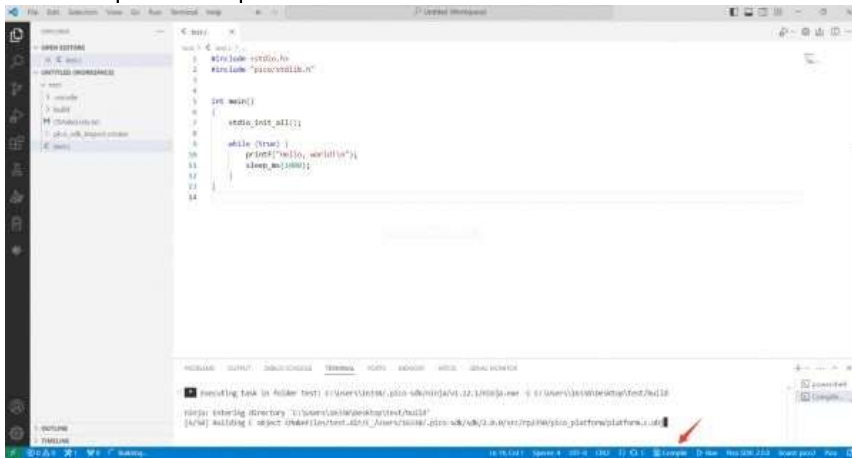
(/wiki/File:Pico-vscode-17.JPG)

8. Select the development board



(/wiki/File:Pico-vscode-18.JPG)

9. Click Compile to compile



(/wiki/File:Pico-vscode-19.JPG)

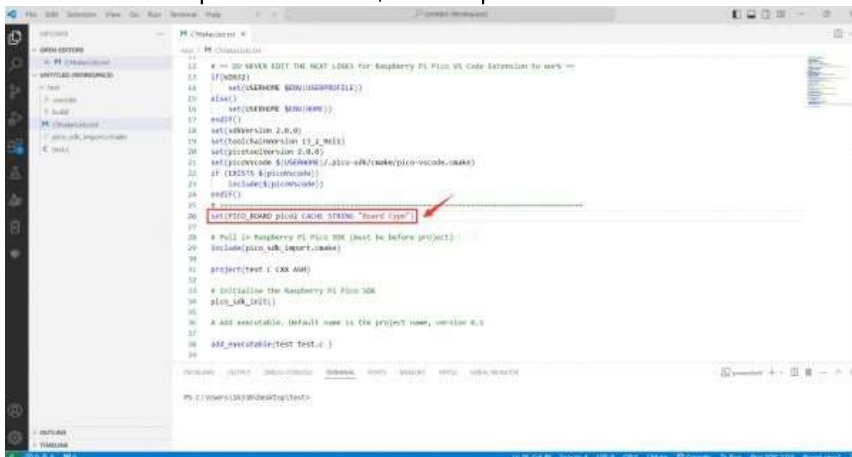
10. The uf2 format file is successfully compiled



(/wiki/File:Pico-vscode-20.JPG)

Import Project

1. The Cmake file of the imported project cannot have Chinese (including comments), otherwise the import may fail
2. To import your own project, you need to add a line of code to the Cmake file to switch between pico and pico2 normally, otherwise even if pico2 is selected, the compiled firmware will still be suitable for pico

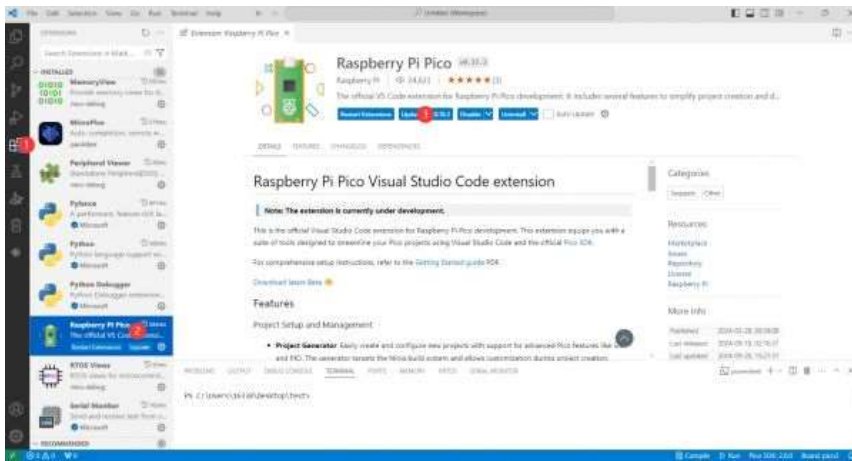


(/wiki/File:Pico-vscode-21.JPG)

set(PICO_BOARD pico CACHE STRING "Board type")

Update Extension

1. The extension version in the offline package is 0.15.2, and you can also choose to update to the latest version after the installation is complete

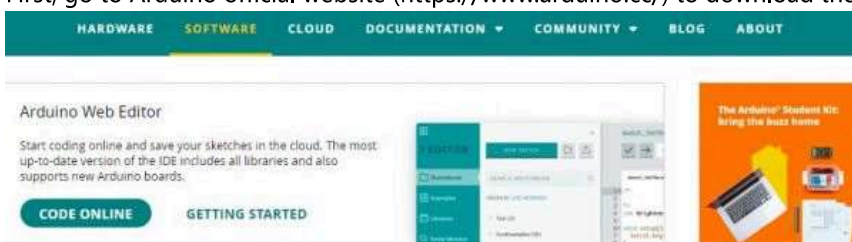


(/wiki/File:Pico-vscode-22.JPG)

Arduino IDE Series

Install Arduino IDE

1. First, go to Arduino official website (<https://www.arduino.cc/>) to download the installation package of the Arduino IDE.



Downloads



(/wiki/File:600px-

Arduino%E4%B8%8B%E8%BD%BD2.0%E7%89%88%E6%9C%AC.jpg)

2. Here, you can select Just Download.

Support the Arduino IDE

Since the release 1.x release in March 2015, the Arduino IDE has been downloaded **69,954,557** times — impressive! Help its development with a donation.

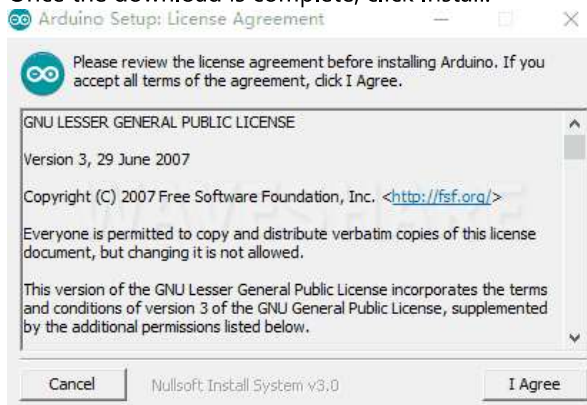
\$3	\$5	\$10	\$25	\$50	Other
-----	-----	------	------	------	-------



Learn more about [donating to Arduino](#).

(/wiki/File:%E4%BB%85%E4%B8%8B%E8%BD%BD%E4%B8%8D%E6%8D%90%E8%B5%A0.png)

3. Once the download is complete, click Install.



(/wiki/File:IDE%E5%AE%89%E8%A3%85%E6%B0%B4%E5%8D%B0-

1.gif)

Notice: During the installation process, it will prompt you to install the driver, just click Install

600px (/w/index.php?

title=Special:Upload&wpDestFile=Arduino%E9%A9%B1%E5%8A%A8%E5%AE%89%E8%A3%85%E6%B0%B4%E5%8D%B0.jpg)

Arduino IDE Interface

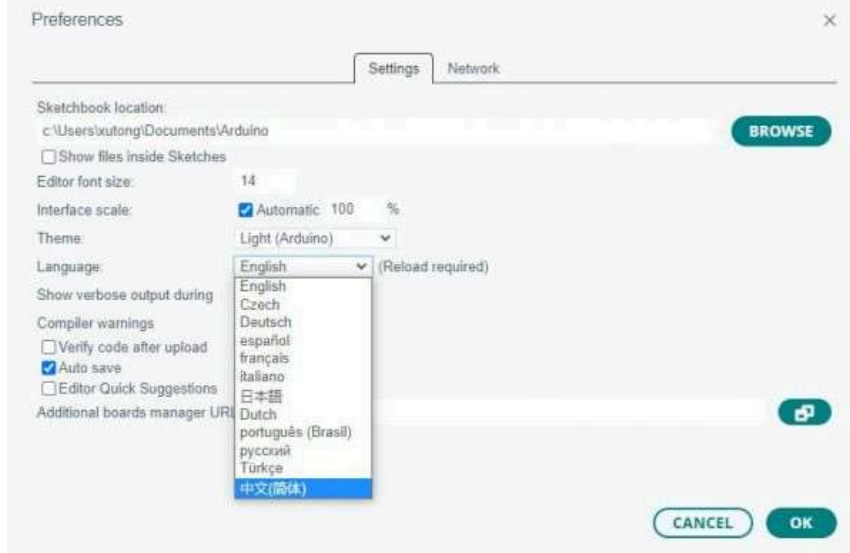
1. After the first installation, when you open the Arduino IDE, it will be in English. You can switch to other languages in File --> Preferences, or continue using the English interface.



(/wiki/File:%E9%A6%96%E9%80%89%E9%A1%B9-

%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87.jpg)

2. In the Language field, select the language you want to switch to, and click OK.



(/wiki/File:600px-

%E9%A6%96%E9%80%89%E9%A1%B9-%E7%AE%80%E4%BD%93%E4%B8%AD%E6%96%87ok.jpg)

Install Arduino-Pico Core in the Arduino IDE

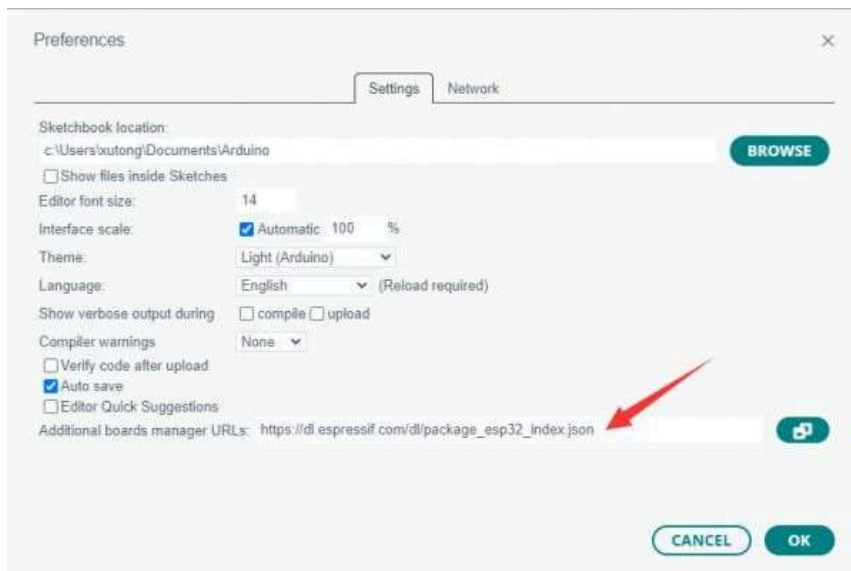
1. Open the Arduino IDE, click on the file in the top left corner, and select Preferences



(/wiki/File:RoArm-M1_Tutorial04.jpg)

2. Add the following link to the attached board manager URL, and then click OK

https://github.com/earlephilhower/arduino-pico/releases/download/4.0.2/package_rp2040_index.json

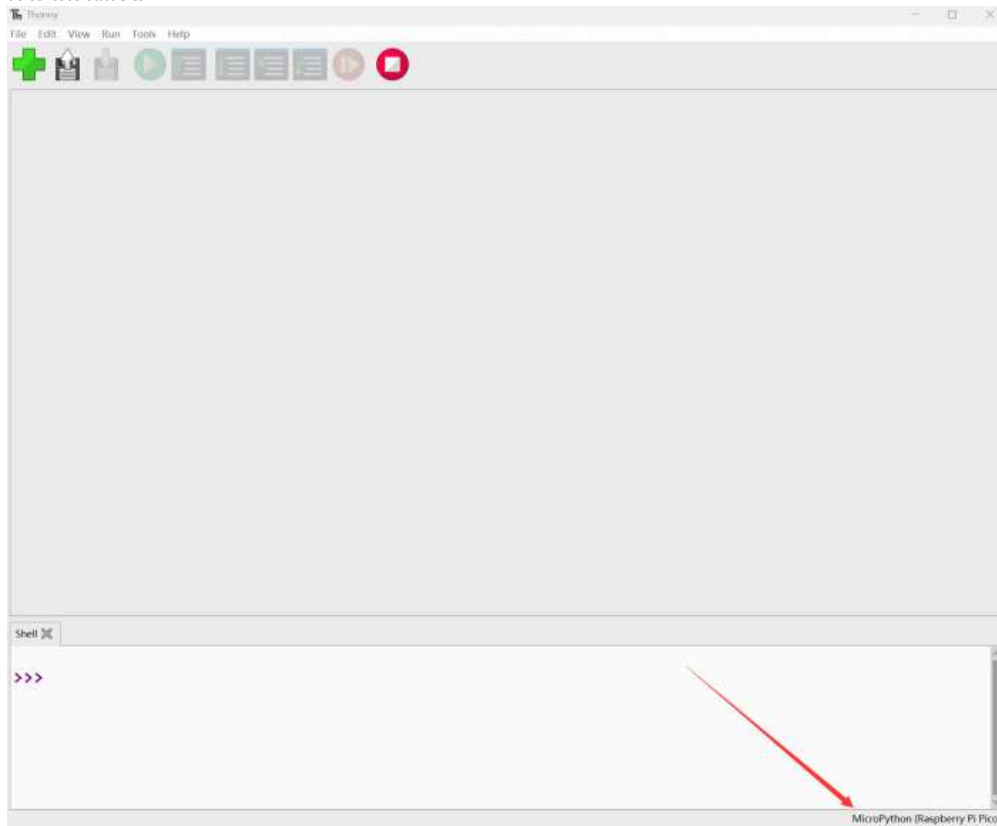


(/wiki/File:RoArm-M1_Tutorial_II05.jpg)

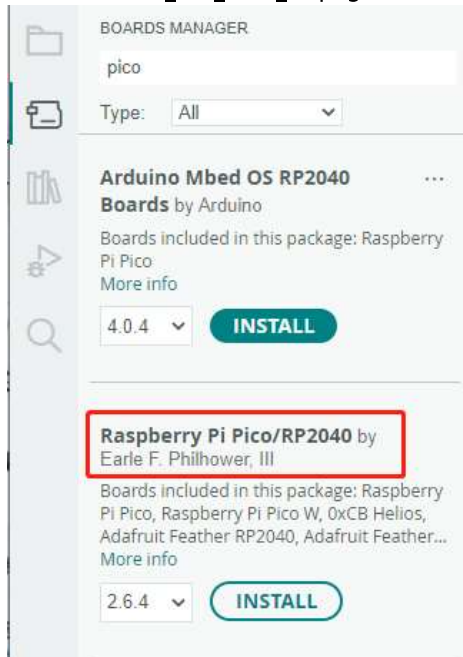
Note: If you already have an ESP32 board URL, you can use a comma to separate the URLs as follows:

`https://dl.espressif.com/dl/package_esp32_index.json,https://github.com/earlephilhower/arduino-pico/releases/download/4.0.2/package_rp2040_index.json`

3. Click Tools > Development Board > Board Manager > Search pico, as my computer has already been installed, it shows that it is installed



(/wiki/File:Pico_Get_Start_05.png)



(/wiki/File:Pico_Get_Start_06.png)

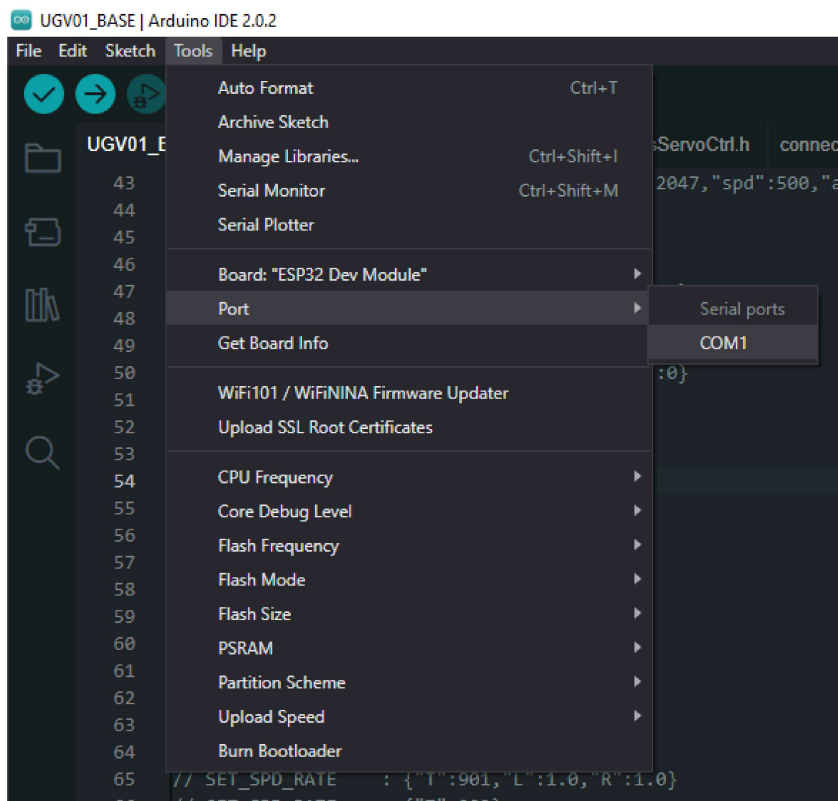
Upload Demo at the First Time

1. Press and hold the BOOTSET button on the Pico board, connect the pico to the USB port of the computer via the Micro USB cable, and release the button after the computer recognizes a removable hard disk (RPI-RP2).



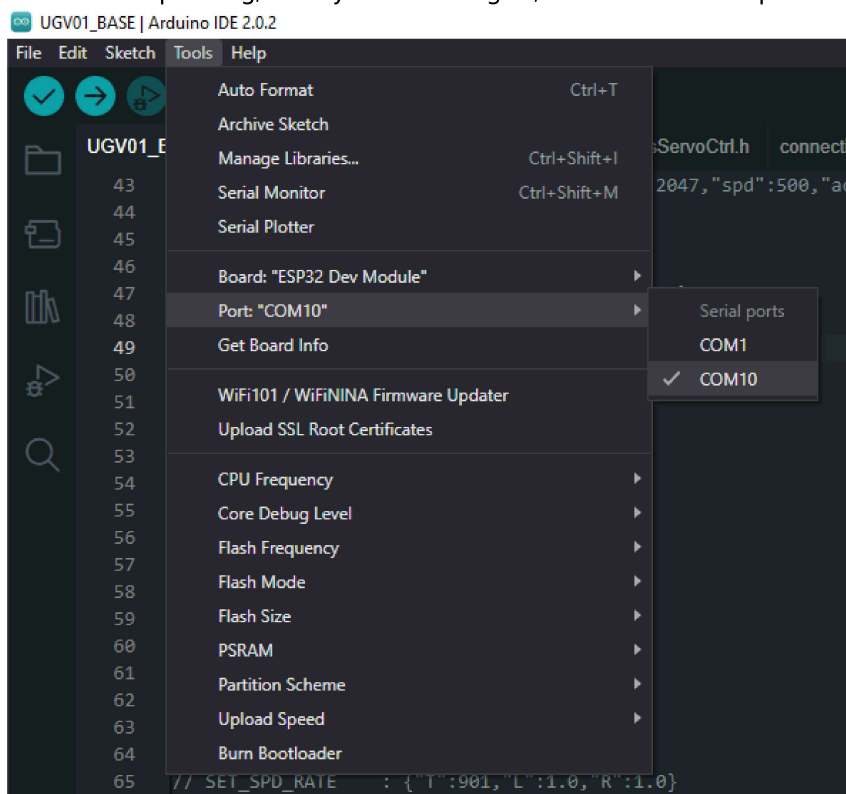
(/wiki/File:Pico_Get_Start.gif)

2. Download the program and open D1-LED.ino under the arduino\PWM\D1-LED path
3. Click Tools --> Port, remember the existing COM, do not click this COM (the COM displayed is different on different computers, remember the COM on your own computer)



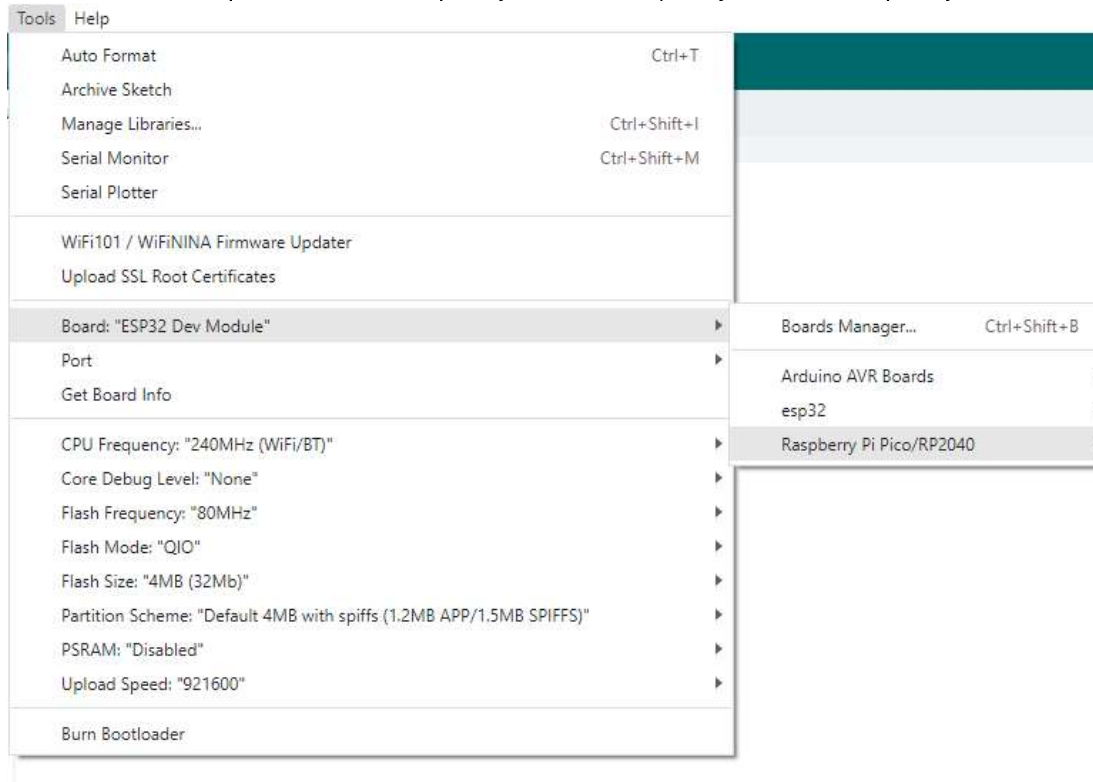
(/wiki/File:UGV1_doenload02EN.png)

4. Connect the driver board to the computer using a USB cable. Then, go to Tools > Port. For the first connection, select uf2 Board. After uploading, when you connect again, an additional COM port will appear



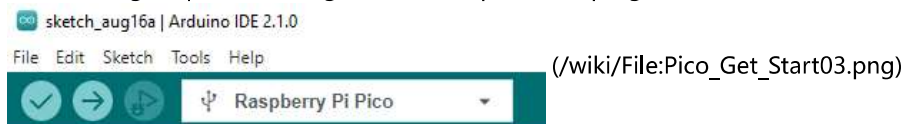
(/wiki/File:UGV1_doenload03EN.png)

5. Click Tools > Development Board > Raspberry Pi Pico > Raspberry Pi Pico or Raspberry Pi Pico 2



(/wiki/File:Pico_Get_Start02.png)

6. After setting it up, click the right arrow to upload the program



(/wiki/File:Pico_Get_Start03.png)

- If issues arise during this period, and if you need to reinstall or update the Arduino IDE version, it is necessary to uninstall the Arduino IDE completely. After uninstalling the software, you need to manually delete all contents within the C:\Users\[name]\AppData\Local\Arduino15 folder (you need to show hidden files to see this folder). Then, proceed with a fresh installation.

Open Source Demos

MircoPython video demo (github) (https://github.com/waveshareteam/Pico_MircoPython_Examples)
MicroPython firmware/Blink demos (C) (https://files.waveshare.com/wiki/common/Raspberry_Pi_Pico_Demo.zip)
Raspberry Pi official C/C++ demo (github) (<https://github.com/raspberrypi/pico-examples/>)
Raspberry Pi official micropython demo (github) (<https://github.com/raspberrypi/pico-micropython-examples>)
Arduino official C/C++ demo (github) (<https://github.com/earlephilhower/arduino-pico>)

Support

Technical Support

If you need technical support or have any feedback/review, please click the **Submit Now** button to submit a ticket, Our support team will check and reply to you within 1 to 2 working days. Please be patient as we make every effort to help you to resolve the issue.
Working Time: 9 AM - 6 PM GMT+8 (Monday to Friday)

Submit Now (<https://service.waveshare.com/>)

